Ticket 2 Technology Identification

CS 4273 – Capstone | Fall 2026

Group F: James Cunningham, Colby Frison, Shone George Kutty Renjan, Patrick Ha, Alan Vo

Mentor/Client: Michael Coffman, Rebecca Mauldin

Due: 9/9/26

Table of Contents

Project information 2

Identify Technologies 2

Research Resources 2

Define Features and Documentation (README) 3

Unit Tests (Python, JavaScript, C++) 9

AMEParserTest.py 9

AMEParserTest.js 11

AMEParserTest.cpp 13

CMakeLists.txt (needed for GTest) 15

run_all_tests.py (Testing Harness) 16

Project information

Field	Value
Course	CS 4273 – Capstone | Fall 2026
Ticket	Ticket 2 – Technology Identification
Project	Pilots Guide to Medical Certification
Group ID	Group F
Team members	James Cunningham, Colby Frison, Shone George Kutty Renjan, Patrick Ha, Alan Vo
Product Owner	Colby Frison
Mentor/Client	Micheal Coffman
Date	9/8/26

Identify Technologies

As a group, we decided to use Python, JavaScript, C++, JSON, HTML, CSS, React, Next.js, and Figma as our technologies.

Research Resources

- Python: https://www.w3schools.com/python/
- JavaScript: https://www.w3schools.com/js/
- C++: https://www.w3schools.com/cpp/
- JSON: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/JSON
- HTML: https://www.w3schools.com/html/
- CSS: https://nextjs.org/learn/dashboard-app/css-styling, https://www.w3schools.com/css/
- React: https://nextjs.org/learn/react-foundations
- Next.js: https://nextjs.org/learn
- Figma: https://help.figma.com/hc/en-us/articles/30848209492887-Course-overview-Figma-Design-for-beginners-2025

Define Features and Documentation (README)

Pilots Guide to Medical Certification

CS-4273 Capstone - Fall 2026 | Group F

A searchable web platform that helps pilots understand FAA medical certification: conditions and sub-conditions, likely outcomes (issue, deferral, or denial), deferral paperwork and tests, and how medications affect flight status.

This project continues prior OU Capstone work (Fall 2025 / Spring 2026) and is designed for possible integration with the FAA Office of Communication. Content should stay accurate, easy to find, easy to understand, and encouraging.

Problem

FAA medical guidance is dense and examiner-oriented. Newer pilots often cannot tell what a condition means for their certificate, what an Aviation Medical Examiner (AME) will need, or whether a medication will affect flight status. Applicants with medical histories — including mental health concerns — sometimes assume a piloting career is over before it starts.

Common pain points:

- Finding accurate information among conflicting sources
- Identifying a condition and its related sub-conditions
- Knowing whether guidance points to issue, deferral, or denial
- Understanding deferral paperwork and tests early enough that approval is not delayed
- Checking how medications affect flight status
- Decoding medical terminology in plain language

Goals

- Build a pilot-facing web platform within an **FAA-acceptable framework**
- Make certification information **searchable** and easy to navigate
- Present conditions, outcomes, deferral requirements, and medication guidance clearly
- Ground content in official FAA sources (parsed from AME Guide / OneGuide PDFs) and mentor-approved material
- Emphasize: get healthy first, see personal physicians for concerns, then pursue the medical certificate

What we are building

The core product is **data presentation and search**. Pilots look up what they need and leave with a clear answer — not a multi-step wizard with saved progress.

Area	What pilots get
Search	Find conditions, sub-conditions, medications, and related guidance
Outcomes	See whether guidance maps to **issue**, **deferral**, or **denial**
Deferral readiness	List paperwork, tests, and docs tied to a deferral path
Medications	See how a medication can affect flight status / certification
Content source	Structured data parsed from official PDFs this semester (external DB possible later)

Key feature (Ticket 2)

Condition disposition classification — take AME Guide–style disposition text and label it `ISSUE`, `DEFER`, `DENY`, `SPECIAL ISSUANCE`, or `UNKNOWN`. That label is what the UI can show as a clear outcome after search.

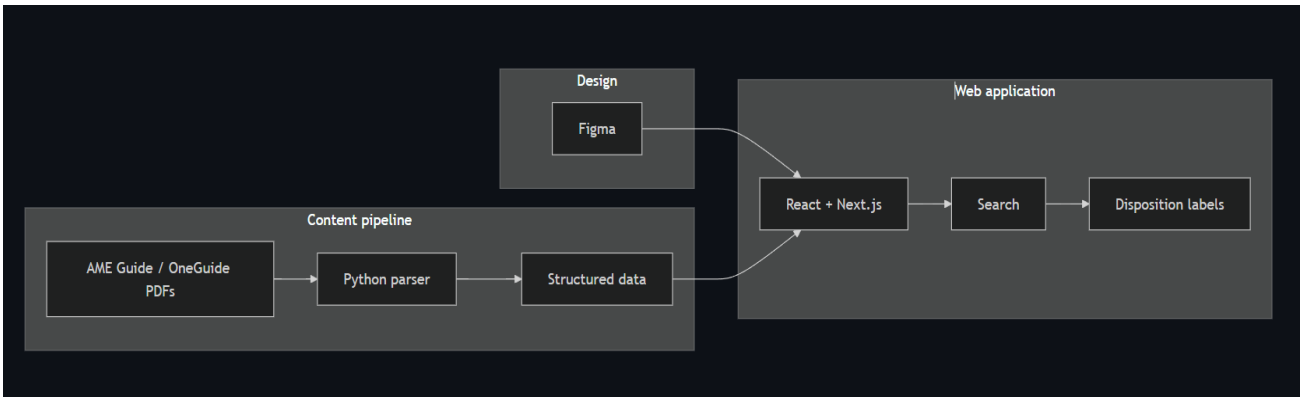
Documented with unit tests in Python, JavaScript, and C++:

- Ticket 2 artifact
- Tests: `AMEParserTest.py` / `.js` / `.cpp` — run with `python Assignments/Ticket_2/src/run_all_tests.py`

Technologies

Technology	Role
Python	PDF parsing, content pipeline, disposition logic, pytest
JavaScript	Web interactivity; disposition helper + Node tests
C++	Ticket 2 third-language unit tests (GoogleTest)
JSON	Structured conditions, dispositions, deferral docs, medication notes
HTML / CSS	Web UI foundation and responsive layout
React + Next.js	Searchable, accessible presentation layer (planned)
Figma	Wireframes and stakeholder review

How they fit together



See Step 1.a — Identify Technologies and Project Context and Requirements.

Repository structure

```
Capstone-Fall2026/
├── README.md
├── Documentation/
│   └── Resources/                # Briefs, proposals, AME Guide
PDFs, requirements
├── Assignments/
│   ├── Ticket_1/                 # Domain understanding
│   └── Ticket_2/
│       ├── instructions/
│       └── src/
│           ├── Ticket2_Artifact.md # Key feature + unit test write-up
│           ├── Step1a_Identify_Technologies.md
│           ├── AMEParserTest.py / .js / .cpp
│           ├── run_all_tests.py   # Run all three language suites
│           ├── AMEParser.py / AMESubset.py  # Pipeline copies for
Ticket 2
│           └── CMakeLists.txt
└── src/
    └── old/           # Prior-semester Python pipeline (kept intact)
```

Getting started

Run Ticket 2 unit tests

Requires Python 3.10+, Node.js 18+, and CMake (for C++).

```
cd Assignments/Ticket_2/src
python run_all_tests.py
```

Content pipeline (prior-semester tools)

Parser copies live under `Assignments/Ticket_2/src/` and `src/old/`. Place the AME Guide PDF where the scripts expect it, install dependencies (`pdfplumber`, `pypdf`), then run `AMEParser.py` on a PDF subset. Details remain in those scripts and the Ticket 2 artifact.

Next.js app

Not scaffolded yet. Setup steps will be added here when the frontend lands.

Progress plan

Phase	Timeline	Deliverables
1. Domain & stack	Weeks 1–2	Ticket 1 domain work; technology selection; README; Ticket 2 unit tests
2. Design	Weeks 2–4	Figma wireframes for search and result views; mentor feedback
3. Content pipeline	Weeks 2–5	Extend PDF parsing; structure conditions, outcomes, deferral docs, medications
4. Frontend prototype	Weeks 4–8	Next.js app with search, disposition labels, deferral and medication info
5. Testing & refinement	Weeks 8–10	Broader unit tests; accessibility review; content accuracy checks
6. Demo & handoff	Weeks 11–15	Presentation, user guide, report; prepare for FAA stakeholder review

Team

Role	Name
Group ID	Group F
Team members	James Cunningham, Colby Frison, Shone George Kutty Renjan, Patrick Ha, Alan Vo
Mentor / Client	Micheal Coffman, Rebecca Mauldin, Jesus J Viramontes

References

- FAA Medical Certification
- FAA Guide for Aviation Medical Examiners
- MedXPress
- AOPA — Preflight your medical
- Project info
- Capstone project proposal
- Project context & requirements
- Ticket 2 artifact

Disclaimer

Educational prototype for CS-4273 Capstone. **Not** an official FAA product. Medical content must be verified against current FAA guidance and mentor-approved sources. The prototype does not collect real personally identifiable information (PII).

Unit Tests (Python, JavaScript, C++)

AMEParserTest.py

```
"""
CS 4273 – Capstone | Fall 2026 | GROUP F

Unit tests for get_decision() — Python port matching AMEParserTest.js / .cpp.

Run with:
  python -m pytest AMEParserTest.py -v
"""

from __future__ import annotations


def get_decision(text: str | None) -> str:
    """
    Pull the disposition keyword out of the raw cell text.
    DEFER/DENY are checked before ISSUE to avoid false matches
    (e.g., 'do not issue' would incorrectly match ISSUE otherwise).
    """
    upper = (text or "").upper()

    if "DEFER" in upper:
        return "DEFER"
    if "DENY" in upper:
        return "DENY"
    if "SPECIAL ISSUANCE" in upper:
        return "SPECIAL ISSUANCE"
    if "ISSUE" in upper:
        return "ISSUE"

    return "UNKNOWN"


# ---------------------------------------------------------------------------
# DispositionSuite
# ---------------------------------------------------------------------------

def test_IssueTest():
```

```
    assert get_decision("AME may issue") == "ISSUE"


def test_DeferTest():
    assert get_decision("Defer to FAA") == "DEFER"


def test_DenyTest():
    assert get_decision("Deny the certificate") == "DENY"


def test_SpecialIssuanceTest():
    assert get_decision("Special Issuance required") == "SPECIAL ISSUANCE"


def test_DeferTakesPriorityOverIssueTest():
    # DEFER is checked before ISSUE, so this should return DEFER
    assert get_decision("Defer, do not issue") == "DEFER"


def test_UnknownDecisionTest():
    assert get_decision("No matching keyword") == "UNKNOWN"


def test_NoneDecisionTest():
    assert get_decision(None) == "UNKNOWN"
    assert get_decision("") == "UNKNOWN"


def test_CaseInsensitiveTest():
    assert get_decision("defer") == "DEFER"


def test_IssueInActionTextTest():
    assert (
        get_decision("ISSUE\nAME may issue and note this in Block 60.") ==
"ISSUE"
    )
```

AMEParserTest.js

```
CS 4273 – Capstone | Fall 2026 | GROUP F

Unit tests for getDecision() — JS port of get_decision() in AMEParser.py.

Run with Node 18+:
  node --test AMEParserTest.js
*/

const { describe, it } = require("node:test");
const assert = require("node:assert/strict");

/**
 * Pull the disposition keyword out of the raw cell text.
 * DEFER/DENY are checked before ISSUE to avoid false matches
 * (e.g., 'do not issue' would incorrectly match ISSUE otherwise).
 *
 * @param {string | null | undefined} text
 * @returns {"DEFER" | "DENY" | "SPECIAL ISSUANCE" | "ISSUE" | "UNKNOWN"}
 */
function getDecision(text) {
  const upper = (text || "").toUpperCase();

  if (upper.includes("DEFER")) {
    return "DEFER";
  }
  if (upper.includes("DENY")) {
    return "DENY";
  }
  if (upper.includes("SPECIAL ISSUANCE")) {
    return "SPECIAL ISSUANCE";
  }
  if (upper.includes("ISSUE")) {
    return "ISSUE";
  }

  return "UNKNOWN";
}

describe("DispositionSuite", () => {
  it("IssueTest", () => {
```

```
    assert.equal(getDecision("AME may issue"), "ISSUE");
  });

  it("DeferTest", () => {
    assert.equal(getDecision("Defer to FAA"), "DEFER");
  });

  it("DenyTest", () => {
    assert.equal(getDecision("Deny the certificate"), "DENY");
  });

  it("SpecialIssuanceTest", () => {
    assert.equal(getDecision("Special Issuance required"), "SPECIAL ISSUANCE");
  });

  it("DeferTakesPriorityOverIssueTest", () => {
    // DEFER is checked before ISSUE, so this should return DEFER
    assert.equal(getDecision("Defer, do not issue"), "DEFER");
  });

  it("UnknownDecisionTest", () => {
    assert.equal(getDecision("No matching keyword"), "UNKNOWN");
  });

  it("NoneDecisionTest", () => {
    assert.equal(getDecision(null), "UNKNOWN");
    assert.equal(getDecision(undefined), "UNKNOWN");
    assert.equal(getDecision(""), "UNKNOWN");
  });

  it("CaseInsensitiveTest", () => {
    assert.equal(getDecision("defer"), "DEFER");
  });

  it("IssueInActionTextTest", () => {
    assert.equal(
      getDecision("ISSUE\nAME may issue and note this in Block 60."),
      "ISSUE",
    );
  });
});
```

```
module.exports = { getDecision };
```

AMEParserTest.cpp

```
/*
CS 4273 – Capstone | Fall 2026 | GROUP F

@brief: Unit tests for get_decision() in AMEParser.py using C++ and GTest.

Execution Steps:
1. Install CMake from https://cmake.org/download/ for your OS
2. Run: cmake -B build
    (NOTE): Above command configures CMake and creates new build folder
3. Run: cmake --build build
    (NOTE): Above command compiles code to an executable and stores in build
folder
4. Run executable:
    - Windows: ./build/Debug/CppUnitTest.exe
    - Mac/Linux: ./build/CppUnitTest
*/

// GTest
#include <gtest/gtest.h>
// Standard Libraries
#include <string>
#include <cctype>

// C++ version of get_decision() in AMEParser.py
std::string get_decision(std::string text)
{
    // convert entire text to uppercase
    for (auto &character : text)
    {
        character = std::toupper(character);
    }

    if (text.find("DEFER") != std::string::npos)
    {
        return "DEFER";
    }
    else if (text.find("DENY") != std::string::npos)
```

```
    {
        return "DENY";
    }
    else if (text.find("SPECIAL ISSUANCE") != std::string::npos)
    {
        return "SPECIAL ISSUANCE";
    }
    else if (text.find("ISSUE") != std::string::npos)
    {
        return "ISSUE";
    }

    return "UNKNOWN";
}

namespace AMEParserTest
{
    TEST(DispositionSuite, IssueTest)
    {
        EXPECT_EQ("ISSUE", get_decision("AME may issue"));
    }

    TEST(DispositionSuite, DeferTest)
    {
        EXPECT_EQ("DEFER", get_decision("Defer to FAA"));
    }

    TEST(DispositionSuite, DenyTest)
    {
        EXPECT_EQ("DENY", get_decision("Deny the certificate"));
    }

    TEST(DispositionSuite, SpecialIssuanceTest)
    {
        EXPECT_EQ("SPECIAL ISSUANCE", get_decision("Special Issuance required"));
    }

    TEST(DispositionSuite, DeferTakesPriorityOverIssueTest)
    {
        // DEFER is checked before ISSUE, so this should return DEFER
        EXPECT_EQ("DEFER", get_decision("Defer, do not issue"));
    }
```

```
    TEST(DispositionSuite, UnknownDecisionTest)
    {
        EXPECT_EQ("UNKNOWN", get_decision("No matching keyword"));
    }

    TEST(DispositionSuite, NoneDecisionTest)
    {
        EXPECT_EQ("UNKNOWN", get_decision(""));
    }

    TEST(DispositionSuite, CaseInsensitiveTest)
    {
        EXPECT_EQ("DEFER", get_decision("defer"));
    }

    TEST(DispositionSuite, IssueInActionTextTest)
    {
        EXPECT_EQ("ISSUE", get_decision("ISSUE\nAME may issue and note this in
Block 60."));
    }
}
```

CMakeLists.txt (needed for GTest)

```
cmake_minimum_required(VERSION 3.14)
project(CppUnitTest CXX)

# module for integrating external dependencies during configuration
include(FetchContent)

# define external dependency we want to integrate (Gtest)
FetchContent_Declare(
  googletest
  GIT_REPOSITORY https://github.com/google/googletest.git
```

```
  GIT_TAG v1.14.0
)

# download Gtest
FetchContent_MakeAvailable(googletest)

add_executable(CppUnitTest AMEParserTest.cpp)
#gtest_main library contains pre-written main function to execute tests
target_link_libraries(CppUnitTest gtest_main)
```

run_all_tests.py (Testing Harness)

```
#!/usr/bin/env python3
"""
CS 4273 – Capstone | Fall 2026 | GROUP F

Run all Ticket 2 disposition unit tests (Python, JavaScript, C++).

Usage (from this directory):
  python run_all_tests.py
"""

from __future__ import annotations

import shutil
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent



def run(label: str, command: list[str], *, cwd: Path = ROOT) -> bool:
    print(f"\n{'=' * 60}")
    print(f"  {label}")
    print(f"  $ {' '.join(command)}")
    print("=" * 60)
    result = subprocess.run(command, cwd=cwd)
    ok = result.returncode == 0
    print(f"\n[{'PASS' if ok else 'FAIL'}] {label}")
    return ok
```

```
def find_cpp_exe(build_dir: Path) -> Path | None:
    candidates = [
        build_dir / "Debug" / "CppUnitTest.exe",
        build_dir / "Release" / "CppUnitTest.exe",
        build_dir / "CppUnitTest.exe",
        build_dir / "CppUnitTest",
    ]
    for path in candidates:
        if path.is_file():
            return path
    return None


def run_python() -> bool:
    test_file = ROOT / "AMEParserTest.py"
    if not test_file.is_file():
        print(f"\n[FAIL] Python - missing {test_file.name}")
        return False
    return run(
        "Python (pytest)",
        [sys.executable, "-m", "pytest", str(test_file), "-v"],
    )


def run_javascript() -> bool:
    node = shutil.which("node")
    if not node:
        print("\n[SKIP] JavaScript - node not found on PATH")
        return False
    test_file = ROOT / "AMEParserTest.js"
    if not test_file.is_file():
        print(f"\n[FAIL] JavaScript - missing {test_file.name}")
        return False
    return run("JavaScript (node --test)", [node, "--test", str(test_file)])


def run_cpp() -> bool:
    cmake = shutil.which("cmake")
    if not cmake:
        print("\n[SKIP] C++ - cmake not found on PATH")
```

```
        return False

    build_dir = ROOT / "build"
    configure_ok = run(
        "C++ configure (cmake)",
        [cmake, "-B", str(build_dir)],
    )
    if not configure_ok:
        return False

    build_ok = run(
        "C++ build (cmake --build)",
        [cmake, "--build", str(build_dir)],
    )
    if not build_ok:
        return False

    exe = find_cpp_exe(build_dir)
    if exe is None:
        print("\n[FAIL] C++ - built executable not found under build/")
        return False

    return run(f"C++ (GTest) - {exe.name}", [str(exe)], cwd=exe.parent)


def main() -> int:
    print("Ticket 2 - running disposition unit tests in 3 languages")
    print(f"Working directory: {ROOT}")

    results = {
        "Python": run_python(),
        "JavaScript": run_javascript(),
        "C++": run_cpp(),
    }

    print(f"\n{'=' * 60}")
    print("  Summary")
    print("=" * 60)
    for name, ok in results.items():
        print(f"  {name:12} {'PASS' if ok else 'FAIL'}")

    all_ok = all(results.values())
```

```
    print(f"\nOverall: {'ALL PASSED' if all_ok else 'SOME FAILED'}")
    return 0 if all_ok else 1


if __name__ == "__main__":
    raise SystemExit(main())
```